🚕

Design Tesla Robotaxi App

功能性需求（Functional Requirements）

1. 乘客（Riders）應該能夠輸入起點與目的地，並取得車資預估（fare estimate）。
2. 乘客應該能夠依據預估車資發出叫車請求（request a ride）。
3. 在收到請求後，系統應該能夠將乘客與附近且可用的自駕車（Autonomous Vehicle, AV）進行配對。
4. 配對成功後，AV 應被派遣（dispatched）並前往接送乘客。

非功能性需求（Non-Functional Requirements）

1. 低延遲配對（Low latency matching）：配對成功或失敗需在 1 分鐘內完成。
2. 強一致性（Strong consistency）：必須避免同一台 AV 被指派給多個行程，或同一個行程被多台 AV 同時指派。
3. 系統需能處理高吞吐量（High throughput），特別是在尖峰時段或特殊活動期間（例如同一個社區同時有 10 萬筆請求）。

buildnoat.org

API Design

- Riders get fare estimates

```
POST /fare
Body: {
  pickupLocation,
  destination
}
-> Fare
```

- Riders request rides

```
POST /rides
Body: {
  fareId
}-> Ride
```

- AV 確認或拒絕乘車的請求。Note ：服務會透過 persistent connection 與 AV 進行通訊，通常使用 RPC。在這裡我們以gRPC 作為介面。

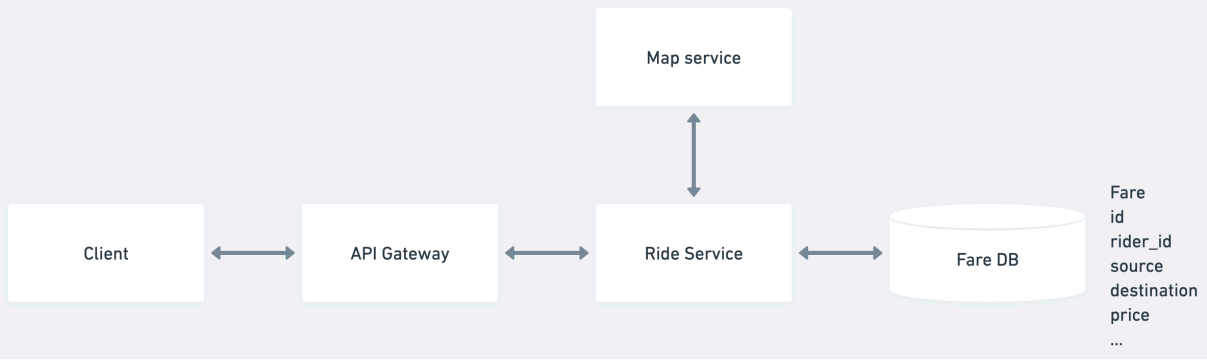
```
message DispatchCommand {
  string ride_id = 1;
  // pickup + destination, etc.
}

message DispatchDecision {
  string ride_id = 1;
  Decision decision = 2; // ACCEPT | REJECT
  string reason = 3; // e.g. "LOW_BATTERY"
}
```

buildmoat.org

High-Level Design

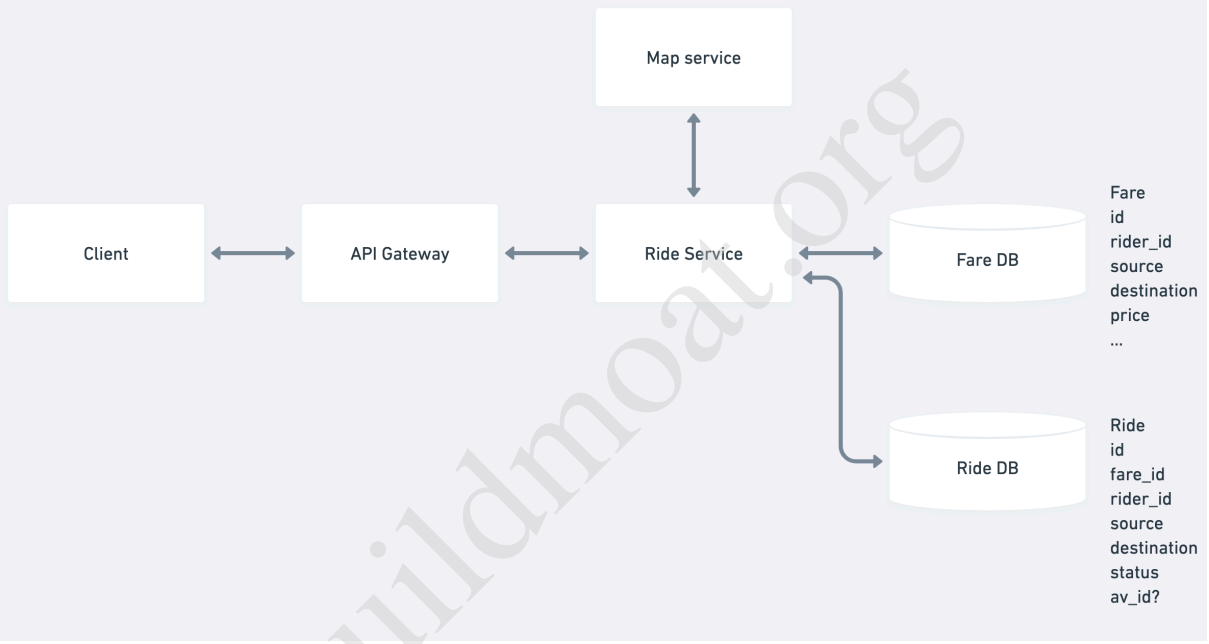
1. 乘客（Riders）應該能夠輸入起點與目的地，並取得車資預估（fare estimate）



1. 乘客透過 POST request，將上車地點（pick up location）與目的地（destination）送至 Ride Service（行程服務）。

2. Ride Service 計算預估車資（estimated fare），並在資料庫中建立一筆紀錄以保存該結果。
    a. Note：車資預估在實務系統中可能相當複雜，通常需要專門的離線資料處理管線（offline data pipeline）與線上服務（online service）來提供即時資料。為了簡化說明，這裡暫時將其視為 Ride Service 內的一個 black box。
3. Ride Service 將新建立的 fare object 回傳給 client。

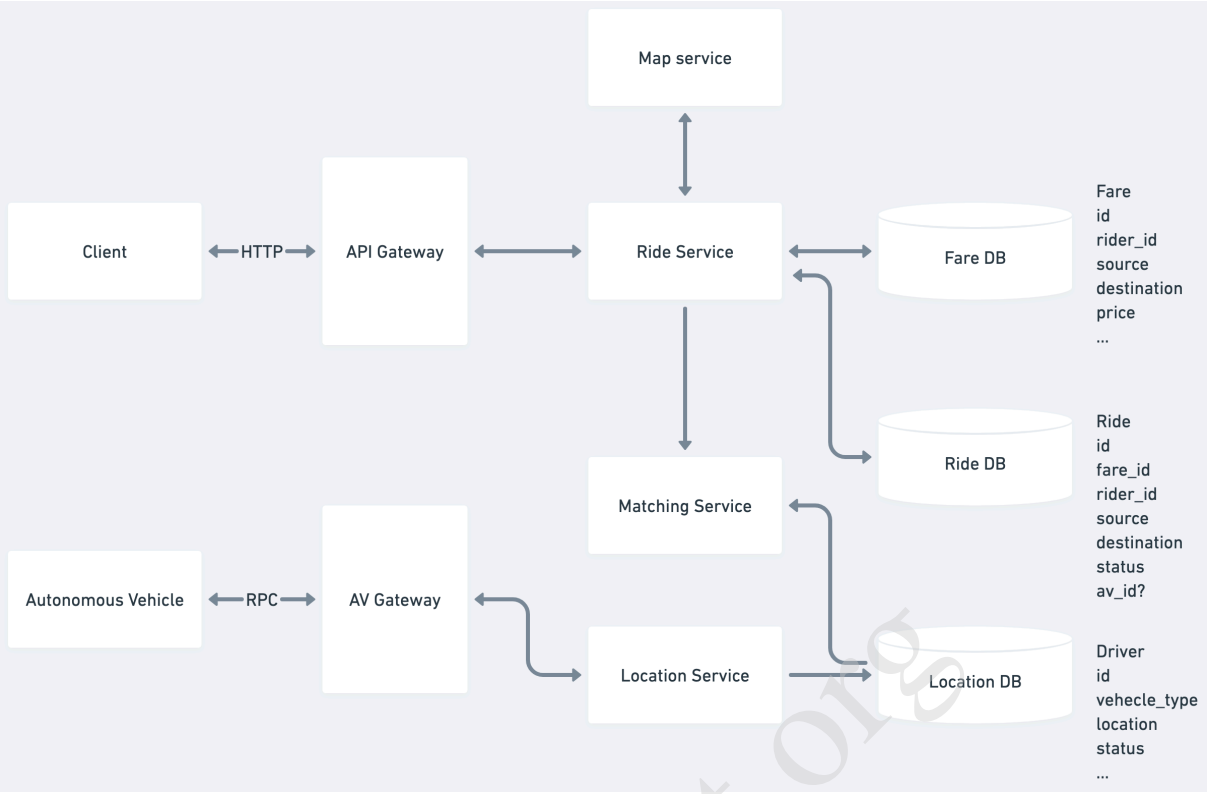
2. 乘客（Riders）應該能夠根據預估車資請求叫車（request a ride）



buildmoat.org

1. 乘客確認報價後，送出另一個 POST request 以確認行程（confirm the ride）。
2. Ride Service 接收叫車請求，並在 ride table 中建立一筆紀錄。
3. Ride Service 接著觸發 matching process，將乘客與一台自駕車（Autonomous Vehicle, AV）進行配對（見下一節）。

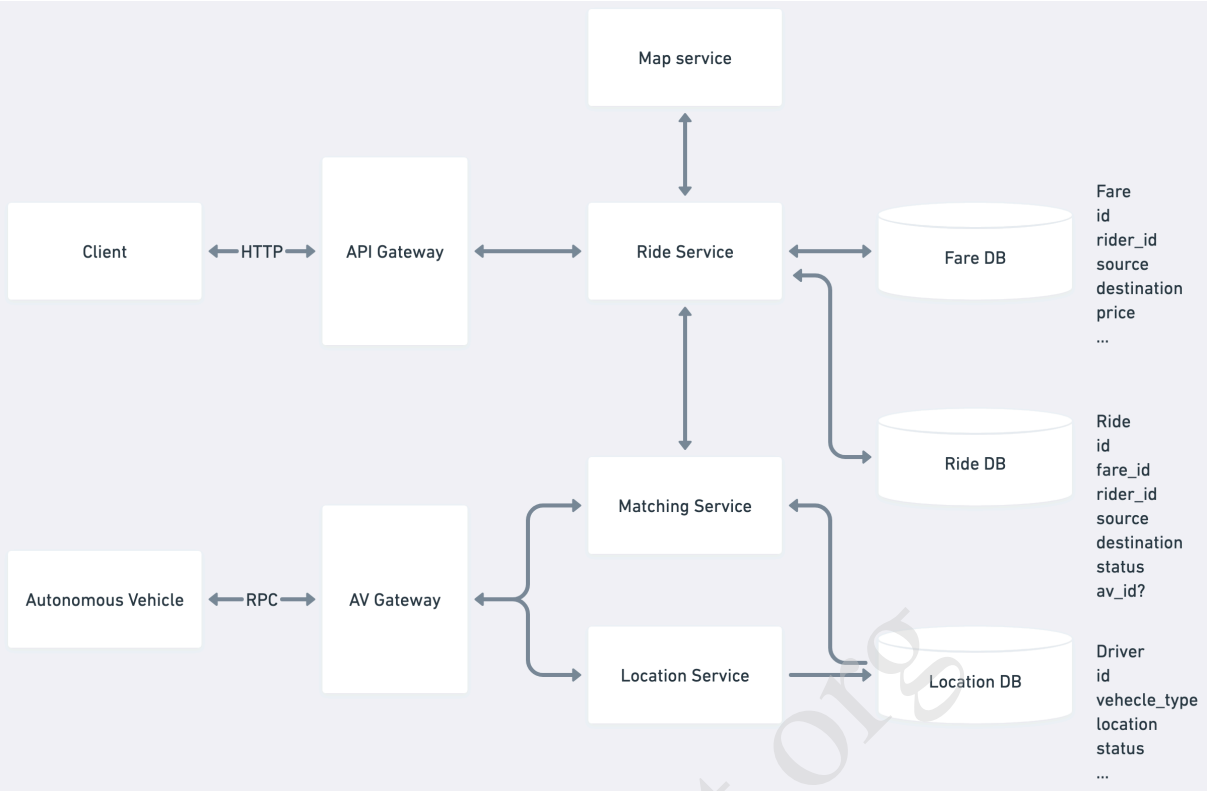
3. 在收到請求後，乘客（Riders）應該能夠與附近且可用的自駕車（Autonomous Vehicle, AV）完成配對



1. 每一台 AV 都會與 AV Gateway（車輛閘道服務）維持一個持久且已驗證的連線（persistent, authenticated connection），例如 gRPC bidirectional stream 或 WebSocket。
    - AV 會透過此連線定期回傳 location 與 status 更新。
    - AV Gateway 會將這些更新轉發至 Location Service（位置服務），由其負責更新 location data。
2. 當 Ride Service 為一筆新行程觸發配對流程時，會向 Matching Service（配對服務）發送 matching request。
3. Matching Service 查詢 Location Store（位置儲存），根據 proximity 與 availability 取得附近可用的 AV 清單，並進行 candidate selection 與 dispatch。

4. 已配對的自駕車（AV）應被派遣並前往接送乘客

buildmoat.org



1. Matching Service 對前一節計算出的 top AV candidate 發出 dispatch command。
2. AV 接受 dispatch command 後，開始 en route to pickup the rider。
    a. 若因 not enough battery power 等原因拒絕行程，Matching Service 會對下一個 candidate 發出 dispatch command。
3. 當 Matching Service 收到 AV 的確認回覆後，會通知 Ride Service 更新資料庫中的 ride status。
4. Ride Service 同時需要通知乘客其行程已成功配對 AV，並提供相關資訊，例如 matched AV 的 real-time location、license plate 等。

buildnoat.org

深入探討（Deep Dives）

1. 我們要如何處理高頻的 AV 位置更新，以及在位置資料上進行高效率的鄰近查詢？

1. **寫入頻率（Frequency of writes）**：

    假設同時有 1,000 萬台自駕車（Autonomous Vehicle, AV）在線，且每台 AV 每 5 秒更新一次位置，這代表大約每秒 200 萬筆更新。這個數字遠遠超過一般資料庫

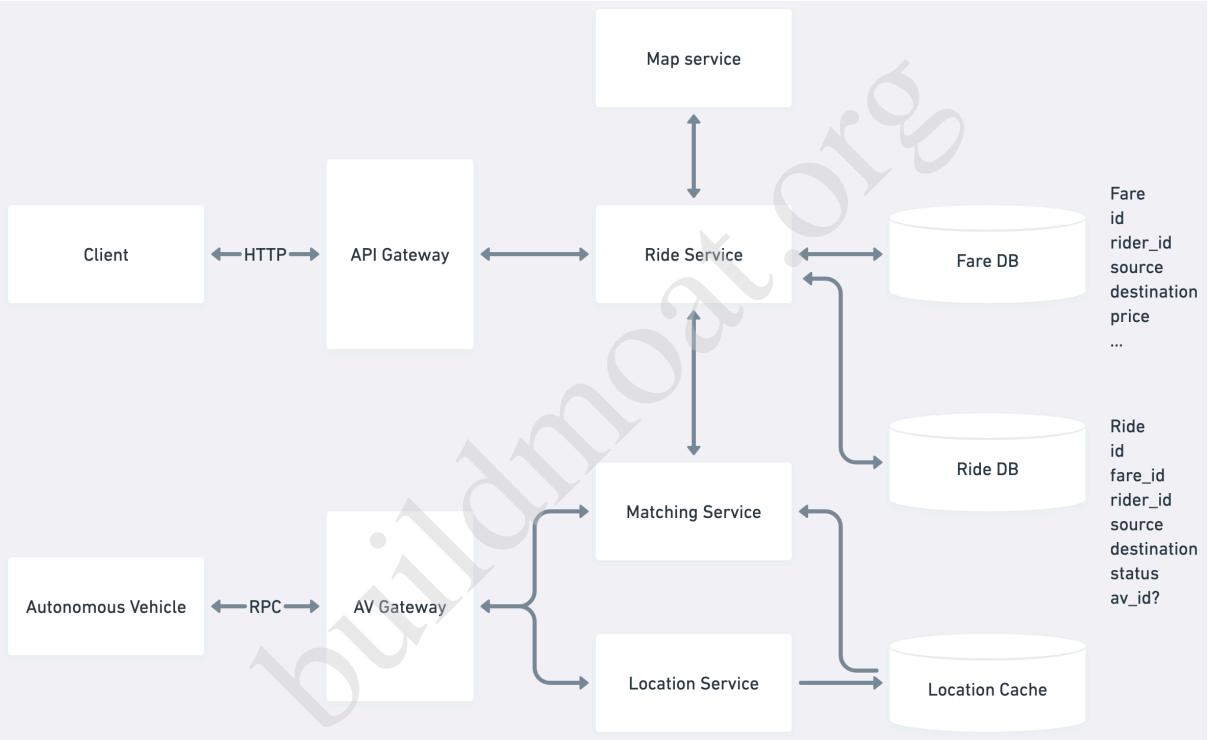
寫入吞吐量的典型上限（~10k/s）。

即使我們可以透過水平擴展（horizontal scaling）將 DB 擴充到足以承受這樣的寫入負載，其成本對多數公司而言也過於昂貴且不合理。

2. **查詢效率（Query efficiency）**：

   若沒有任何優化，要根據經緯度（lat/long）查詢資料，通常需要對整張表進行 full table scan，並計算每位乘客位置與所有 AV 位置之間的距離。

   這在有數百萬使用者時會極度低效。即使對 lat/long 欄位建立索引，傳統的 B-tree index 也不適合處理地理座標這類多維度資料，導致 proximity search 的查詢效能仍然不佳。



由於位置資料是暫時性的（ephemeral），我們不需要保存 AV 的歷史位置，僅需保留「最新位置」即可滿足行程配對需求，因此可以使用 Redis（in-memory store）來儲存 AV 位置並承受高頻位置更新。

此外，Redis 支援多種地理查詢指令（例如 `GEOADD`、`GEOSEARCH`），使得 proximity search 高效且容易實作。

我們需要同時維護以下兩種對應關係：

- `encoded_location → [driver_ids]`
- `driver_id → encoded_location`

如此一來，在位置更新時，才能將 `driver_id` 從舊位置中移除。

Redis 的缺點在於資料存放於記憶體中，服務中斷時存在資料遺失的風險。不過 Redis 提供 persistence solutions，例如 Redis Database（RDB）或 AOF（Append Only File），可將 snapshot 或 log 寫入磁碟，在系統故障後進行資料恢復。

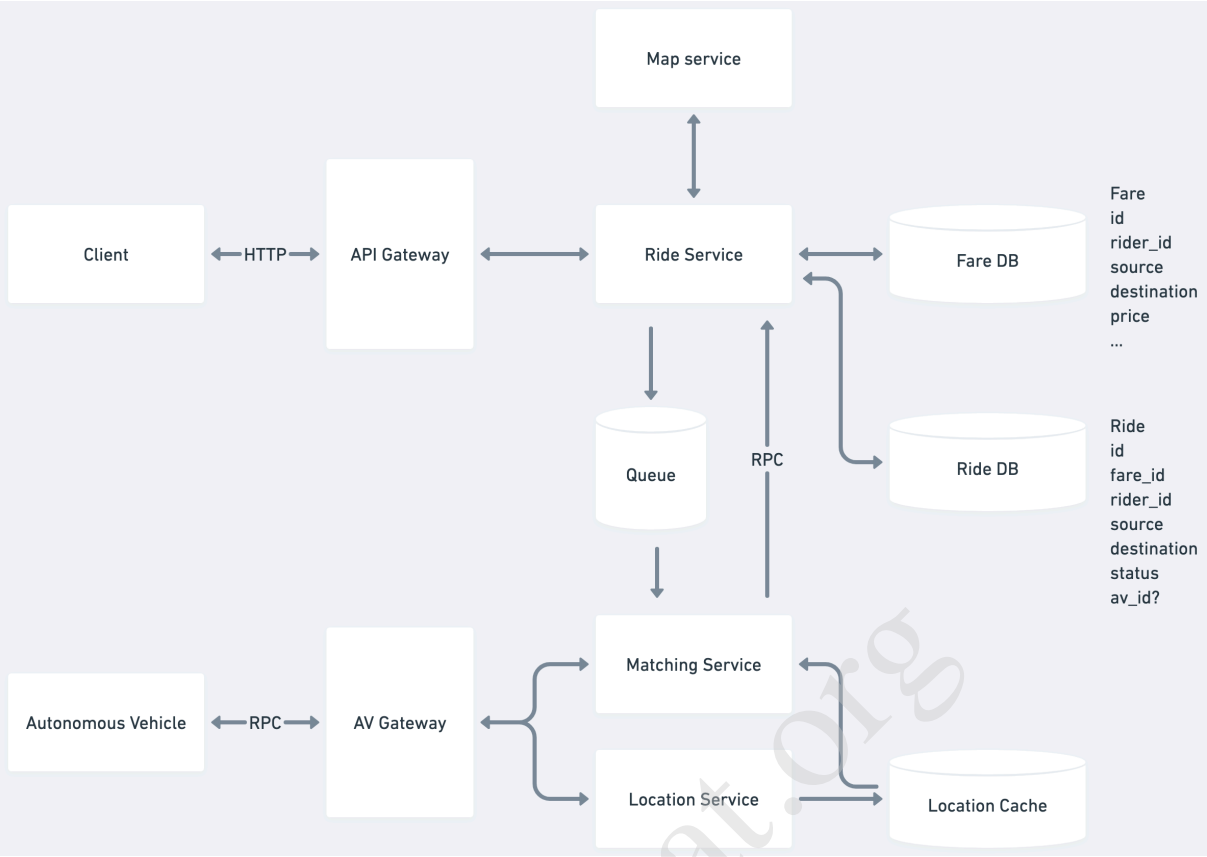
> Note：
>
> 在 1,000 萬台 AV 每 5 秒更新一次位置（約 200 萬 writes/sec）的情境下，單一 Redis instance 無法承受此負載。即使使用 Redis Cluster，也需要大量 shard（通常是數十個）。
>
> 在 Uber / Waymo 等真實世界的 production 系統中，通常會基於 H3 / S2 cell hierarchy 搭配 in-memory storage，自行實作客製化的 geospatial indexing service。
>
> 不過，在系統設計面試中，使用 Redis GEO commands 已是完全可接受的解法，因為重點在於 matching architecture，而非地理索引的底層優化。

2. 我們要如何確保在尖峰需求期間，不會遺失任何叫車請求？

在尖峰需求期間，系統可能同時收到大量 ride requests，進而導致請求被丟棄。這在大型活動或假期期間尤其嚴重。此外，我們也需要避免 Matching Service 的 instance crash 或 restart，導致行程遺失。

為了在 Ride Service（RS）與 Matching Service（MS）之間加入緩衝層（buffer）以吸收流量高峰，我們引入一個 queue（佇列）：

buildmeat.org



1. 當 Ride Service 收到使用者確認的叫車請求後，只需將事件發佈到 queue，並立即完成 request。換言之，matching process 的執行結果不會影響 Ride Service 的 availability。
2. Matching Service 從 queue 中消費 ride events，取得 AV candidates 清單，依序通知候選車輛，取得其中一台的確認後，最後再通知 Ride Service 更新 DB 中對應的 ride state。
    a. 在 MS → RS 的通訊上，我們選擇使用 RPC，因為在 retry loop 中處理同步回饋（例如更新失敗時，嘗試 dispatch 下一個 candidate）會比較容易實作。

3. 我們要如何避免同一筆行程被多台 AV 指派？

在 matching-dispatching flow 中，Matching Service（MS）會先從 location cache 計算出一組候選 AV，並依序嘗試 dispatch。

若某個 candidate 拒絕或 timeout，MS 便會嘗試下一個。那如果第一台 AV 的 dispatch request timeout，MS 已 dispatch 給第二台 AV，但第一台與第二台的 ACCEPT 回應同時到達，該怎麼辦？

在分散式系統中，為了提升可擴展性（horizontal scalability），服務通常設計為 stateless。也就是說，Matching Service 會有多個 instance（workers），且發送 dispatch request 與接收 accept response 的 instance 可能不同。

buildmoat.org

基於此，我們將 matching state 也儲存在 Redis，讓所有 workers 能共享狀態。matching state 的 K-V 結構如下：

```
Key: match:ride:<ride_id>
Value:{
    "candidates": [v1, v2, v3, ...],
    "cursor": 0/1/2...,
    "status": "SEARCHING" | "DONE" | "FAILED"
}
```

1. Matching Service 的某個 worker 從 location cache 計算候選清單，並在 Redis 中建立一筆 matching state，cursor 初始化為 0。
2. 若 candidate[0] 失敗（REJECT 或 TIMEOUT），另一個 worker 可以重新讀取 matching state，遞增 cursor，並 dispatch 下一個 candidate。
3. 為避免 contention（多個 worker 同時更新同一筆 state），我們對每個 ride 使用一個 per-ride lock，確保同一時間只有一個 worker 能更新該狀態。
4. 當某個 worker 收到 AV 的 ACCEPT 回應時，必須先取得 lock，並確認狀態仍為 `SEARCHING`，才能將狀態更新為 `DONE`。
5. 透過步驟 3 與 4，我們可以保證只有一個 worker 能成功將狀態設為 `DONE`，因此同一筆 ride 只會被指派給一台 AV。

4. 我們要如何避免同一台 AV 被指派給多筆行程？

延續前一節的 matching state 設計，假設不同 rides 的候選清單出現重疊，例如：

```
R1 ->[V1, V2, V3]
R2 ->[V1, V3, V5]
```

若兩個 ride 的 worker 同時 dispatch 給 V1，且因為網路問題或 AV bug，V1 同時接受了 R1 與 R2，該如何處理？

這裡必須依賴 Ride DB 的 ACID 特性，來保證 **1 AV : 1 Ride**。其中一種做法是在 Ride table 上建立 unique index：

```
CREATE UNIQUE INDEX uniq_active_ride_per_driver
ON rides(av_id)
WHERE statusIN ('DRIVER_ASSIGNED', 'IN_PROGRESS');
```

buildnoat.org

這個 unique index 確保所有狀態為 `DRIVER_ASSIGNED` 或 `IN_PROGRESS` 的 rides，其 `av_id` 必須是唯一的。

因此，若 Ride Service 中有兩個 worker 嘗試將不同 rides 指派給同一個 `av_id`，只有其中一個 update 會成功。失敗的 worker 則可以發出事件，表示指派失敗，並觸發重新配對（re-match）流程。

buildmoat.org